- NAME OF THE TEAM- CODESTORM

- ***Problem Statement***:We address a grid-based pathfinding problem in a dynamic and hazardous environment inspired by Wumpus World.The environment is an N × N grid containing:
- Wumpus (moving hazards)
- Pits (high penalty zones)

Breeze cells (moderate penalty)

Time zones (time reduction zones)

The agent starts from a start cell and must reach a goal cell.

PROBLEM STATEMENT:

Grid navigation with dynamic hazardsObjective: minimize time safely

- 2. ApproachAlgorithm Used:Modified Bellman ford Algorithm
- Why:Weighted edges (time costs)Dynamic state changes
- 3. State RepresentationExplain:(r, c, time, wumpus_positions, stench_count, visited)
- 4. Wumpus Movement LogicFrom your code:Horizontal oscillationBased on row parityTwo-phase movement (A/B)
- 5. ConstraintsNo backtrackingMax 2 stench allowedDynamic Wumpus collisionTime penalties: Pit: +5
- Breeze: +2
- Timezone: -3

- 6. Optimization TechniquesPriority Queue (Min Heap)State pruningMemoization (_wCache)Lexicographically smallest path
- 7. VisualizationExplain UI:Grid systemEntity placementAnimation engineAgent log system
- 8. ResultsInclude:SAFE caseUNSAFE casePath output format
- 9. ComplexityTime: ~O(states × log states)Space: O(states)

Objective:

1. Find a safe path with minimum total time, while:
2. Avoiding lethal conditions (Wumpus, excessive stench)
3. Handling dynamic changes due to moving hazards
4. Respecting constraints like no backtracking

- ***Approach***
- -- Algorithm Used:

Modified Bellman-Ford Algorithm (State-based adaptation)

Why Bellman-Ford?

We chose Bellman-Ford because:

1. Supports weighted edges, including:

   Positive weights (pit +5, breeze +2)

   Negative weights (time zone −3)
2. Handles dynamic state transitions
3. Works in multi-constraint environments

Unlike BFS (unweighted) and standard Dijkstra (static graphs), our approach adapts to:

Changing Wumpus positions

State-dependent constraints (stench count, visited cells)

- ***CORE IDEA:***

- *We treat each step as a state transition problem, where:*

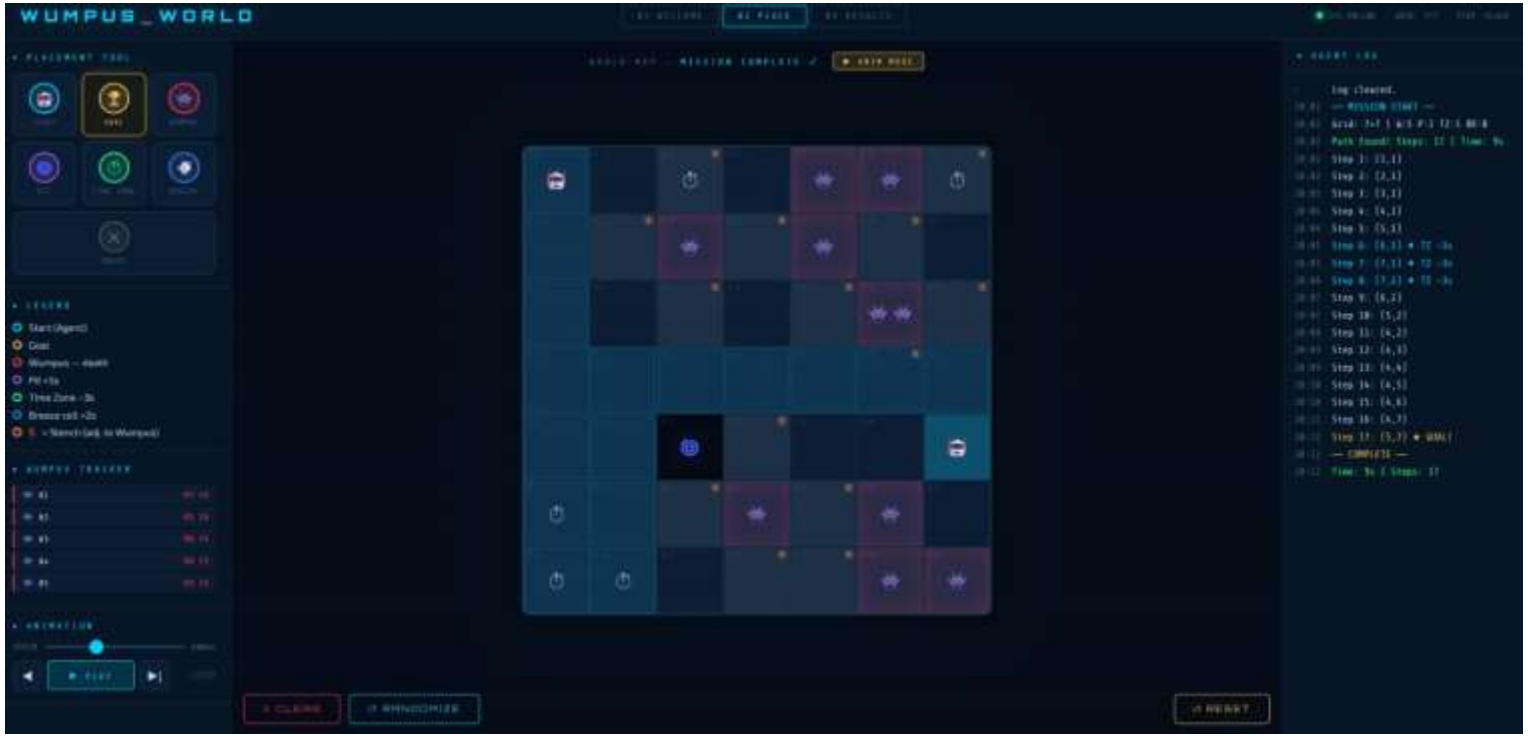
- Each move:
- Updates agent position
- Updates Wumpus positions
- Updates cost (time)
- We iteratively relax states similar to Bellman-Ford
- Bellman-Ford algorithm is preferred over Dijkstra's algorithm in scenarios where the graph contains negative edge weights. Dijkstra's algorithm assumes that once the shortest path to a node is found, it cannot be improved further, which is only valid when all edge weights are non-negative. However, in the presence of negative weights, a shorter path may appear later, making Dijkstra unreliable. Bellman-Ford overcomes this by repeatedly relaxing all edges, ensuring correct shortest path computation even with negative values.

- Another major advantage of Bellman-Ford is its ability to detect negative weight cycles. In graphs where a cycle has a total negative weight, the shortest path is undefined because repeatedly traversing the cycle keeps reducing the total cost indefinitely. Bellman-Ford identifies such cycles by performing an additional relaxation step and checking for further updates. This makes it highly useful in systems where detecting invalid or exploitable paths is important.

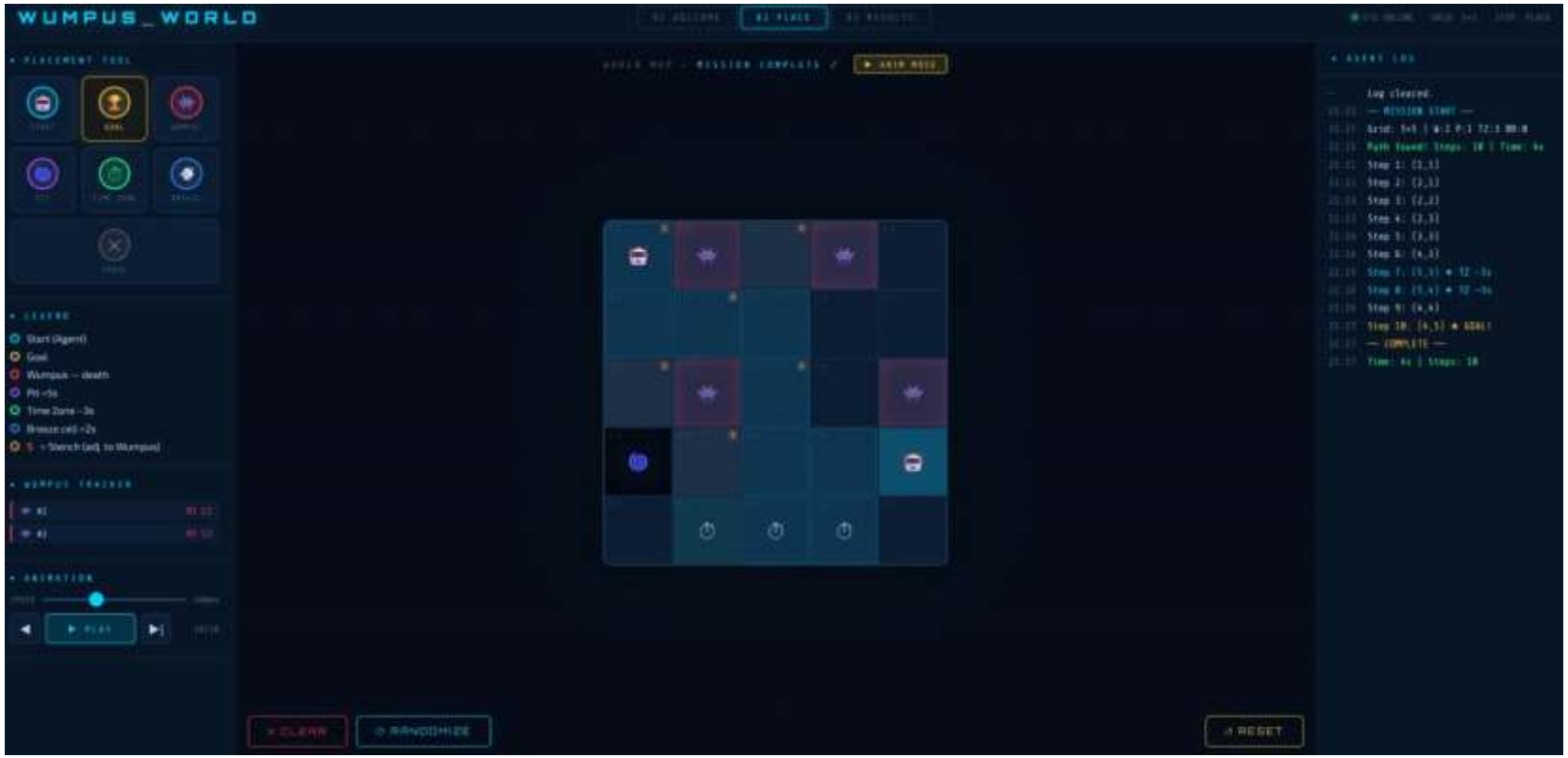
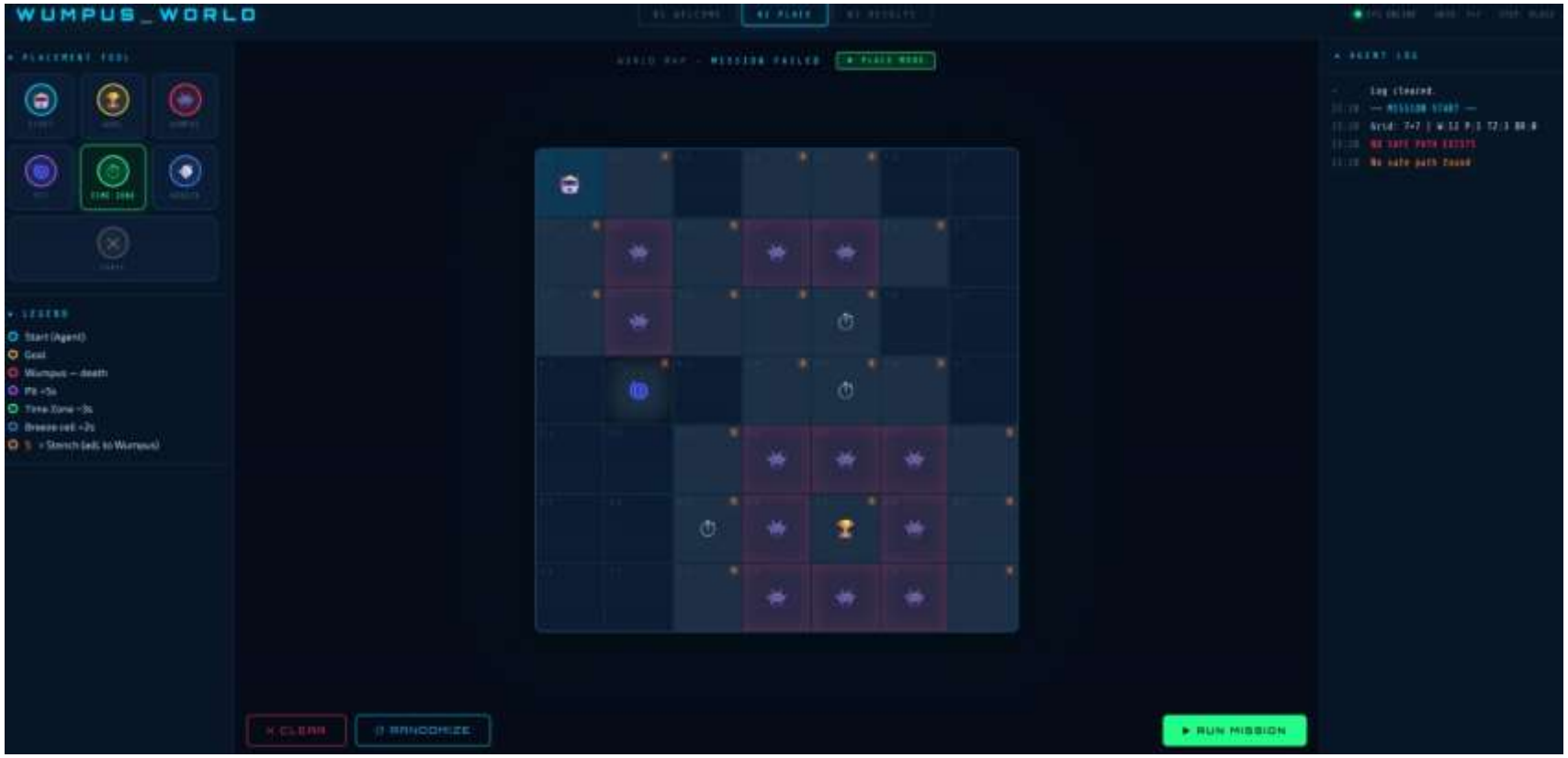
- Bellman-Ford is also more suitable for dynamic or state-dependent environments. Unlike Dijkstra's algorithm, which follows a greedy approach and finalizes nodes early, Bellman-Ford continuously updates distances over multiple iterations. This allows it to adapt to changing conditions such as moving obstacles, time-based costs, or evolving states. In problems like grid navigation with dynamic hazards, this flexibility ensures that the computed path remains valid under changing conditions.

Dashboard/UI Walkthrough

Test case 1



Test case 2



Test case 3